

Experiment 6

React SPA with User Data

App.js

```
// Program 6: SPA with routing (Home, Users), useEffect + useState,
// conditional rendering and a loading indicator.
// Offline: the "public API" is replaced by a local function that returns
// data after a delay (same behaviour as fetch, but no internet).
// Needs: npm install react-router-dom (one-time). Usage: save as src/App.jsx
import React, { useState, useEffect } from "react";
import { BrowserRouter, Routes, Route, NavLink } from "react-router-dom";

// ----- Fake API (replaces https://jsonplaceholder.typicode.com/users) -----
const USERS_DATA = [
  { id: 1, name: "Leanne Graham", email: "leanne@example.com", city: "Hyderabad" },
  { id: 2, name: "Ervin Howell", email: "ervin@example.com", city: "Chennai" },
  { id: 3, name: "Clementine Bauch", email: "clem@example.com", city: "Mumbai" },
  { id: 4, name: "Patricia Lebsack", email: "patricia@example.com", city: "Delhi" },
  { id: 5, name: "Chelsey Dietrich", email: "chelsey@example.com", city: "Pune" },
];

const fetchUsers = (shouldFail = false) => {
  new Promise((resolve, reject) => {
    setTimeout(() => {
      shouldFail ? reject(new Error("Failed to load users")) : resolve(USERS_DATA);
    }, 2000); // simulate 2-second network delay
  });
};

// ----- Pages -----
const Home = () => (
  <div>
    <h2>Home</h2>
    <p>Open the Users page to see data loaded with useEffect.</p>
  </div>
);

function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetchUsers()
      .then((data) => setUsers(data))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, []); // runs once when the page mounts

  // Conditional rendering
  if (loading) return <p>Loading users...</p>;
  if (error) return <p style={{ color: "red" }}>{error}</p>;

  return (
    <div>
      <h2>Users</h2>
      <table border="1" cellPadding="8" style={{ borderCollapse: "collapse" }}>
        <thead>
          <tr><th>ID</th><th>Name</th><th>Email</th><th>City</th></tr>
        </thead>
        <tbody>
          {users.map((u) => (
            <tr key={u.id}>
              <td>{u.id}</td><td>{u.name}</td><td>{u.email}</td><td>{u.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

// ----- App -----
export default function App() {
  const link = ({ isActive }) => ({
    marginRight: 15, fontWeight: isActive ? "bold" : "normal",
  });
  return (
    <BrowserRouter>

```

Experiment 6

React SPA with User Data

App.js (Contd.)

```
<nav style={{ padding: 12, background: "#eee" }}>
  <NavLink to="/" style={link}>Home</NavLink>
  <NavLink to="/users" style={link}>Users</NavLink>
</nav>
<div style={{ padding: 20 }}>
  <Routes>
    <Route path="/" element={Home} />
    <Route path="/users" element={Users} />
  </Routes>
</div>
</BrowserRouter>
);
}
```

Experiment 6

Output

[Home](#) [Users](#)

Users

ID	Name	Email	City
1	Leanne Graham	leanne@example.com	Hyderabad
2	Ervin Howell	ervin@example.com	Chennai
3	Clementine Bauch	clem@example.com	Mumbai
4	Patricia Lebsack	patricia@example.com	Delhi
5	Chelsey Dietrich	chelsey@example.com	Pune

Experiment 6

React SPA with User Data

Aim

1. To develop a React single-page application with routing between Home and Users.
2. To load user data using `useEffect` and `useState`, render it conditionally, and show a loading indicator.

Procedure

1. Create Home and Users pages and configure routing.
2. Define/load user data and manage it with `useState`.
3. Use `useEffect` to fetch/load the data when the Users page mounts.
4. Use conditional rendering for loading/error/data states.

Sample Input

- Open Users and wait while the user data is loaded.

Sample Output

- The Users page displays user ID, name, email and city after the loading state.

Result

Thus, a React SPA with Home and Users routing, state, effect-based loading and conditional rendering was successfully developed.